University of Exeter

ECM2433: The C Family

C preprocessor: cpp

Preprocessor directives

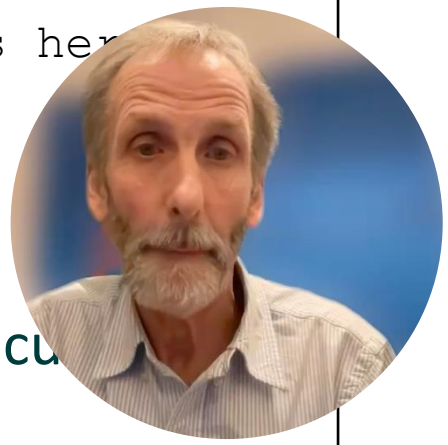
Each source file is pre-processed by the C preprocessor

- Rewrites file with macro expansion

```
#include <stdio.h>
int main ()
{
    /* some code goes here */
    return 0;
}
```

cpp

```
/* contents of stdio.h here*/
int main ()
{
    /* some code goes her
    return 0;
}
```



#include <stdio.h>: find the file called stdio.h and copy its contents into the cu
position in this program, replacing this line.

Preprocessor example

```
/* cpp-eg.c*/
#include "myheader.h"

#define MAGIC 42

int main(int argc, char **argv) {

    float k = MAGIC * PI;

    bar(13, 26);
}
```

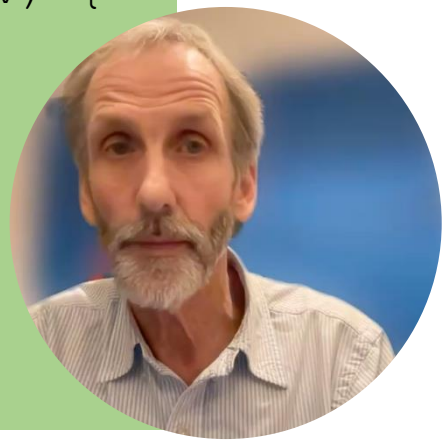
```
/* myheader.h */

#define PI 3.1412659

int foo(int, float);

#include "another.h"
```

```
/* another.h */

void bar(int, int)
```

Preprocessor example

```
$ cpp cpp-eg.c
# 1 "cpp-eg.c"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 399 "<built-in>" 3
# 1 "<command line>" 1
# 1 "<built-in>" 2
# 1 "cpp-eg.c" 2

# 1 "./myheader.h" 1

int foo(int, float);
```

```
# 1 "./another.h" 1
void bar(int, int);

# 6 "./myheader.h" 2
# 2 "cpp-eg.c" 2




int main(int argc, char **argv) {

    float k = 42 * 3.1412659;

    bar(13, 26);
}
```



#include: <> versus " "

```
#include <stdio.h>
#include "myFile.h"
```

<stdio.h>: search for this file using the path in which the pre-processor expects to find such files

"myFile.h": search for this file firstly in the same directory as this program, and then using the path in which the pre-processor expects to find such files. Search path can be specified with `-I` compiler flag.

```
$ ls -l /usr/include/std*
-rw-r--r-- 1 root root  1629 Sep 29 20:02 /usr/include/stdc-predef.h
-rw-r--r-- 1 root root  8135 Sep 29 20:02 /usr/include/stdint.h
-rw-r--r-- 1 root root  2806 Sep 29 20:02 /usr/include/stdio_ext.h
-rw-r--r-- 1 root root 31641 Sep 29 20:02 /usr/include/stdio.h
-rw-r--r-- 1 root root 34030 Sep 29 20:02 /usr/include/stdlib.h
```

```
$ find /usr/include -name "stddef.h"
/usr/include/linux/stddef.h
```

#define

```
#define TRUE  1
#define FALSE 0
#define ELEPHANT "Size of an elephant!"
```

Define a macro and replace any instances of the macro name with its definition.

```
printf(ELEPHANT);
printf("This is %d\n", TRUE);
```

cpp

```
printf("Size of an elephant!");
printf("This is %d\n", 1);
```

but not in quoted strings:

```
printf("It is an ELEPHANT");
```

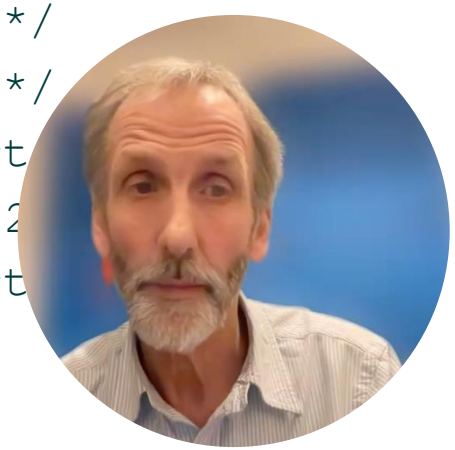
```
printf("It is an ELEPH
```

#define

```
#include <math.h>
```

Contains:

```
# define M_El          2.718281828459045235360287471352662498L /* e */
# define M_LOG2El      1.442695040888963407359924681001892137L /* log_2 e */
# define M_LOG10El     0.434294481903251827651128918916605082L /* log_10 e */
# define M_LN2l        0.693147180559945309417232121458176568L /* log_e 2 */
# define M_LN10l       2.302585092994045684017991454684364208L /* log_e 10 */
# define M_PIl         3.141592653589793238462643383279502884L /* pi */
# define M_PI_2l       1.570796326794896619231321691639751442L /* pi/2 */
# define M_PI_4l       0.785398163397448309615660845819875721L /* pi/4 */
# define M_1_PIl       0.318309886183790671537767526745028724L /* 1/pi */
# define M_2_PIl       0.636619772367581343075535053490057448L /* 2/pi */
# define M_2_SQRTPIl   1.128379167095512573896158903121545172L /* 2/sqrt
# define M_SQRT2l      1.414213562373095048801688724209698079L /* sqrt(2
# define M_SQRT1_2l    0.707106781186547524400844362104849039L /* 1/sqrt
#define M_PI
```



Macros with #define

University of Exeter

#define can be used to create macros:

```
#define ADD(x, y)       x+y

Z = ADD(12, n)
```

results in

```
Z = 12+n
```

But care needs to be taken:

1. Definitions are substituted before the compiler compiles the source code, so a
   error introduced is hard to trace.

2. The direct substitution can lead to unexpected results.

#define: unexpected results

University of Exeter

Consider the following definition:

```
#define MAX(a,b) a > b ? a : b
```

and this piece of code:

```
int i = MAX(2,3) + 5;
int j = MAX(3,2) + 5;
```

The macro substitution leads to this:

```
int i = 2 > 3 ? 2 : 3+5;
int j = 3 > 2 ? 3 : 2+5;
```

#define: unexpected results

University of Exeter

Consider the following definition:

```
#define MAX(a,b) a > b ? a : b
```

and this piece of code:

```
int i = MAX(2,3) + 5;
int j = MAX(3,2) + 5;
```

The macro substitution leads to this:

```
int i = 2 > 3 ? 2 : 3+5;
int j = 3 > 2 ? 3 : 2+5;
```

#define: unexpected results

University of Exeter

Consider the following definition:

```
#define MAX(a,b) a > b ? a : b
```

and this piece of code:

```
int i = MAX(2,3) + 5;
int j = MAX(3,2) + 5;
```

The macro substitution leads to this:

```
int i = 2 > 3 ? 2 : 3+5;
int j = 3 > 2 ? 3 : 2+5;
```

The result is : `i = 8`

#define: unexpected results

University of Exeter

Consider the following definition:

```
#define MAX(a,b) a > b ? a : b
```

and this piece of code:

```
int i = MAX(2,3) + 5;
int j = MAX(3,2) + 5;
```

The macro substitution leads to this:

```
int i = 2 > 3 ? 2 : 3+5;
int j = 3 > 2 ? 3 : 2+5;
```

The result is : `i = 8` and `j = 3`, while we should expect `i = j = 8`!

#define: unexpected results

University of Exeter

Brackets are often the answer:

```
#define MAX(a,b) ((a)>(b)?(a):(b))
```

and this piece of code:

```
int i = MAX(2,3) + 5;
int j = MAX(3,2) + 5;
```

now becomes this:

```
int i = ((2)>(3)?(2):(3))+5;
int j = ((3)>(2)?(3):(2))+5;
```

resulting in `i=8` and `j=8`. This will even work with something like

```
int i = MAX(x+y,z) + 5;
int j = MAX(3.98-2,6.234) + 5;
```

but ...

#define: unexpected results

As before:

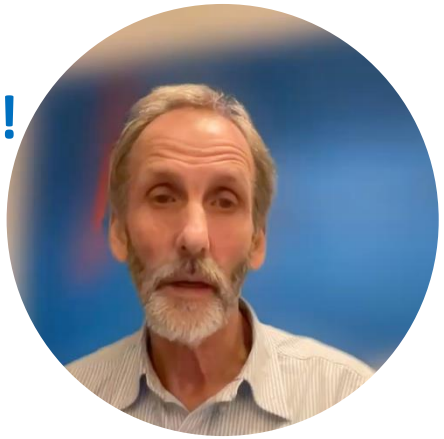
```
#define MAX(a,b) ((a)>(b)?(a):(b))
```

What about this:

```
int i = 2;
int j = 3;
int k = MAX(i++, j++);
```

The outcome is: `k=4, i=3` and `j=5`!

So don't use #define in this way; write proper functions instead!



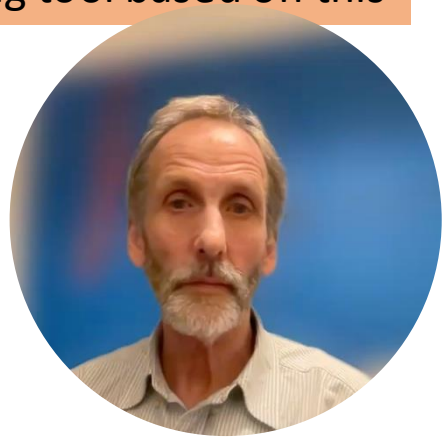
Stringification: convert macro argument to a string

`#ARGUMENT` expands to a valid C string constant containing ARGUMENT.

```
#include <stdio.h>

#define REPORT_float(FOO) fprintf(stderr, "%s is %f\n", #FOO, FOO)


int main() {

    float pi = 3.1415926;
    float *p = &pi;

    REPORT_float(pi);
    REPORT_float(*p + 3);

    return 0;
}
```

pi is 3.141593
*p + 3 is 6.141593

See https://github.com/chunqian/icecream-c for a useful debugging tool based on this



Selective compilation

University of Exeter

#ifdef	if a symbol is defined
#ifndef	if a symbol is not defined
#else	otherwise
#elif	else if
#endif	end of the #ifdef or #ifndef block
#undef	undefine a macro or symbol

```
#ifndef __UNIX__
/* some C code in here for non-UNIX systems */
#else
/* some C code in here for UNIX systems*/
#endif
```

Useful predefined macros

__FILE__	the current file name
__LINE__	the current line number in this file
__DATE__	the date this object code was generated from the source
__TIME__	the time this object code was generated from the source
__STDC__	1 if this implementation of C conforms to the ANSI/ISO standard; any other value means not

```
#include <stdio.h>
int main ()
{
   printf("Compile date is %s, time is %s\n",__DATE__,__TIME__);
   printf("Error in line %d of program %s\n",__LINE__,__FILE__);
#ifdef __STDC__
   printf("ANSI C\n");
#else
   printf("Not ANSI C\n");
#endif
   return 0;
}
```

```
Compile date is Feb 07 2026, time is 18:43:15
Error in line 5 of program macros.c
ANSI C
```

Macro names

- Informal rule: macro name is all capitals
  C has many informal rules; this one makes your code easier to follow
- System macros start with two underscores
  so that they do not interfere with your own macros, and so that you can recognise them

Here is a valid C program:

```
#include <stdio.h>
#define STARTMAIN int main () {
#define ENDMAIN return 0; }

STARTMAIN
   printf("Hello World!\n");
ENDMAIN
```

Summary

C preprocessor directives

- #include – include other files
  - giving your programs access to useful library functions and values
- #define – macro expansion
  - macros to make your code more readable (and perhaps faster)
  - macros for values that might change some time in the future
- #ifdef -- conditional compilation
  - controlling what code gets compiled